Deep Learning

08-06-26

Diminishing learning rate:

In Adagrad :

$$w_{t+1} = w_t - \eta_t g_t$$

η_t ↓ adaptive learning rate

$$\alpha_t = \alpha_{t-1} + (g_t)^2$$

$$\alpha_t = \sum_{i=1}^{t} (g_t)^2$$

α_t - sum of squares of gradients till step t

as number of steps t increase:

$t \uparrow \quad \alpha_t \uparrow \quad \eta_t \downarrow$

$$\eta_t = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$

At the start of model training, η_t is larger and makes model learn faster and as number of steps increase - η_t becomes smaller and makes model training slower

- diminishing learning rate → causes slower model training → less updation in weights

Adadelta Optimizer :

$$\alpha_t = \beta \alpha_{t-1} + (1-\beta)(g_t)^2$$

β is +ve constant 0.9, 0.95, 0.85

Due to multiplying factor β, α_t won't become too large as number of steps t increase

↓

this solves issue of diminishing learning rate in very large ANN models

$$\eta_t = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$

$$w_{t+1} = w_t - \eta_t g_t$$

Adagrad

$$\alpha_t = \alpha_{t-1} + (g_t)^2$$

$$\eta_t = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$

$$w_{t+1} = w_t - \eta_t g_t$$

↓

diminishing learning rate

Adadelta

$$\alpha_t = \beta \alpha_{t-1} + (1-\beta)(g_t)^2$$

$$\eta_t = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$

$$w_{t+1} = w_t - \eta_t g_t$$

↓

no diminishing learning rate

Adam Optimizer:

Adam = Adaptive learning rate + momentum

$$V_t = \beta V_{t-1} + (1-\beta) g_t$$ - momentum

$$\alpha_t = \gamma \alpha_{t-1} + (1-\gamma)(g_t)^2$$

$$\eta_t = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$ - adaptive learning rate

$$W_{t+1} = W_t - \eta_t \cdot V_t$$

$$\boxed{W_{t+1} = W_t - \frac{\eta}{\sqrt{\alpha_t + \varepsilon}} \cdot V_t}$$

Adam is the most preferred optimizer in deep learning use cases.

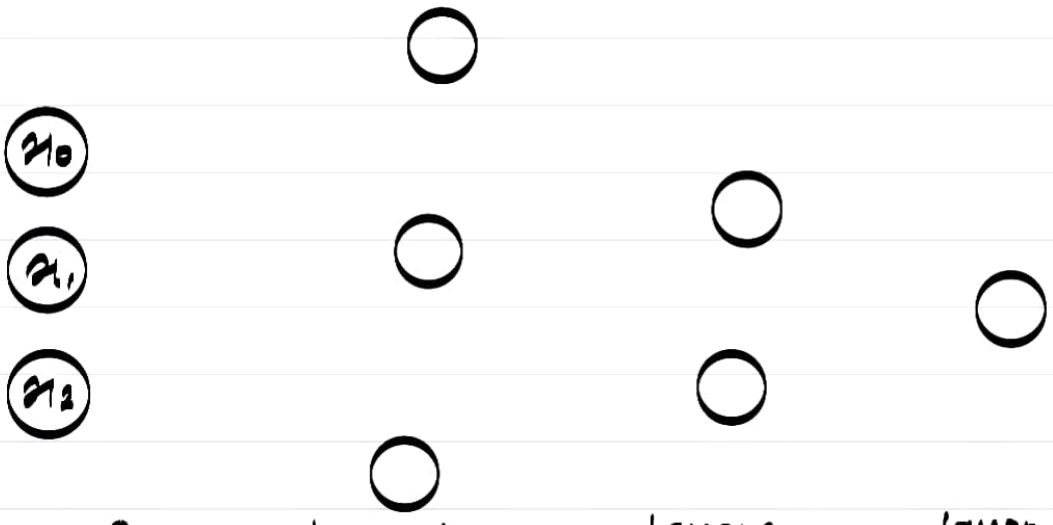
Adam has 3 major parameters

$\beta \rightarrow$ for momentum

$\gamma \rightarrow$ for adaptive LR

$\eta \rightarrow$ initial learning rate

ANN : Binary classification model



No. of hidden layers + no. of nodes in each hidden layer + activations in hidden layers

- these are the major hyperparameters in this model

As this is Binary clf:

- last layer / output layer must have single node
- last layer must have sigmoid activation function

in layer 1:

$$z^1 = w^1 a^0 = w^1 x$$

$$a^1 = f_1(z^1)$$

in layer 2:

$$z^2 = w^2 a^1$$

$$a^2 = f_2(z^2)$$

In layer 3 / output layer :

$$z^3 = w^3 a^2$$

$$a^3 = \text{sigmoid}(z^3)$$

for a given data point x

$$a^3 = \text{sigmoid}(z^3)$$

$$a^3 \in [0, 1]$$

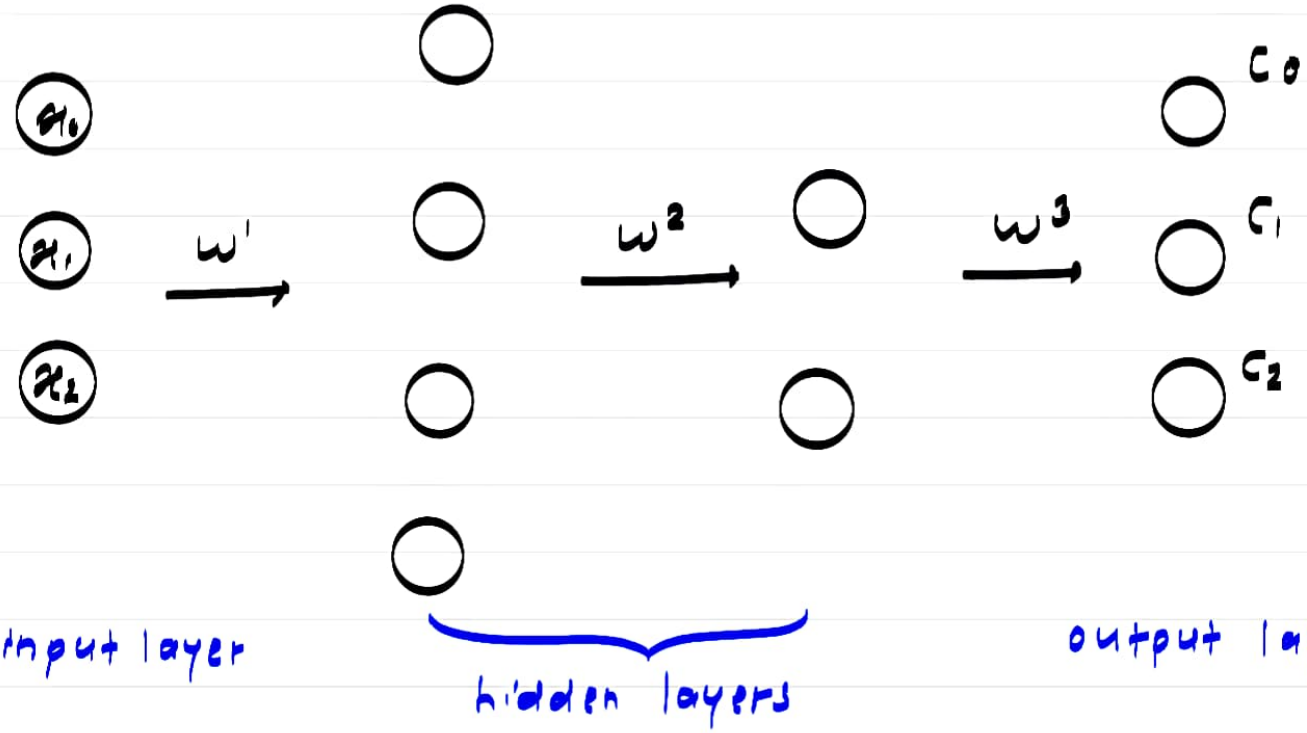
in binary clf: $\hat{y}$ can be 0 or 1

a^3 = probability of $\hat{y} = 1$ given x as input

$$a^3 = p(\hat{y} = 1 \mid x)$$

datapoints	$p(\hat{y} = 1 \mid d)$	threshold = 0.75
d_1	$p(\hat{y}_1 = 1 \mid d_1) = 0.82$	$\hat{y}_1 = 1$
d_2	$p(\hat{y}_2 = 1 \mid d_2) = 0.41$	$\hat{y}_2 = 0$
d_3	$p(\hat{y}_3 = 1 \mid d_3) = 0.58$	$\hat{y}_3 = 0$
d_4	$p(\hat{y}_4 = 1 \mid d_4) = 0.90$	$\hat{y}_4 = 1$

ANN: Multiclass classification model



As this is multiclass classifier:

- no. of nodes in output layer must be equal to no. of unique classes in target variable
  - each node will correspond to one of the classes

- Activation function used in output layer must be softmax

in layer 3 / output layer:

$$z^3 = w^3 a^2$$

z^3: $3 \times n$; w^3: 3×2 ; a^2: $2 \times n$

n - no. of datapoints

z^3 has 3 outputs for every datapoint

$$a^3 = \text{softmax}(z^3)$$

a^3: $3 \times n$ — z^3: $3 \times n$

$$z^3 \rightarrow z_0^3 \quad z_1^3 \quad z_2^3$$

$$\downarrow \quad \downarrow \quad \downarrow$$

$$c_0 \quad c_1 \quad c_2$$

$$z_0^3 \longrightarrow \text{softmax} \longrightarrow e^{z_0^3} / \Sigma e^z = a_0^3$$

$$z_1^3 \longrightarrow \text{softmax} \longrightarrow e^{z_1^3} / \Sigma e^z = a_1^3$$

$$z_2^3 \longrightarrow \text{softmax} \longrightarrow e^{z_2^3} / \Sigma e^z = a_2^3$$

where

$$\Sigma e^z = e^{z_0^3} + e^{z_1^3} + e^{z_2^3}$$

for any datapoint d

$a_0^3 = p(\hat{y} = c_0 \mid d)$

$a_1^3 = p(\hat{y} = c_1 \mid d)$

$a_2^3 = p(\hat{y} = c_2 \mid d)$

} final prediction $= \text{argmax}(a_0^3, a_1^3, a_2^3)$

	a_0^3	a_1^3	a_2^3	$\hat{y}$
d_1	0.2	0.2	(0.4)	$\hat{y}_1 = c_2$
d_2	(0.6)	0.3	0.1	$\hat{y}_2 = c_0$
d_3	0.2	(0.7)	0.1	$\hat{y}_3 = c_1$

In binary clf model: Binary cross entropy loss

$$L = -\sum_{i=1}^{n} y_i \cdot \log(p_i) + (1-y_i) \cdot \log(1-p_i)$$

$p_i = \text{sigmoid}(z^3)$

$p_i = a^3 \rightarrow$ output from the last layer

In multiclass clf model: Cross entropy loss

	y	$\hat{y}$			p_i
a_1	c_0	a^3_{01}	a^3_{11}	a^3_{21}	$p_1 = a^3_{01}$
a_2	c_1	a^3_{02}	a^3_{12}	a^3_{22}	$p_2 = a^3_{12}$
a_3	c_2	a^3_{03}	a^3_{13}	a^3_{23}	$p_3 = a^3_{23}$

$$L = -\sum_{i=1}^{n} y_i \cdot \log(p_i)$$

$n \rightarrow$ no. of datapoints

$$L = -\sum_{i=1}^{3} y_i \cdot \log(p_i)$$

$$L = -[1 \times \log(a^3_{01}) + 1 \times \log(a^3_{12}) + 1 \times \log(a^3_{23})]$$

Overfitting in ANN:

ANN model complexity increases as
- no. of layers increase
- no. of nodes within layer increase

As complexity increases → overfiting also increases

To avoid overfiting, we apply regularization in ANN models

L1 / L2 regularization:

modify loss function of ANN

$$L = MSE + \lambda \sum_{i=1}^{k} |w_i| \quad - L1$$

$$L = MSE + \lambda \sum_{i=1}^{k} (w_i)^2 \quad - L2$$

k → no. of weights in ANN across all layers

Binary clf

$$L = BCE + \lambda \sum_{i=1}^{k} |w_i| \quad - L1$$

$$L = BCE + \lambda \sum_{i=1}^{k} (w_i)^2 \quad - L2$$